

IIT GANDHINAGAR

CS202 – Software Tools and Techniques for CSE

Assignment 1

Nishit Mistry (22110172)

Contents

Laboratory Session 1	3
Introduction	3
Tools	3
Setup	3
Local Environment Preparation	3
Global Git Configuration	4
Methodology and Execution	4
Creating the Local Repository	4
First Add and Commit	4
Creating the Remote on GitHub	4
Verifying History	4
Cloning	4
Python Code and Linting Preparation	5
Setting Up GitHub Actions for Pylint	6
Observing CI Feedback	7
Common Pylint Issues Resolved	7
Results and Analysis	8
Discussion and Conclusion	8
Repository Link	8
Laboratory Session 2	9
Introduction	9
Tools	9
Setup	9
Installing PyDriller	9
Methodology and Execution	10
Bug-Fixing Commit Identification	10
Diff Extraction	11
Commit Message Prediction with LLM	13
Rectifier Formulation	15
Evaluation of Research Questions	16
Results and Analysis	19
Conclusion	19
Repository Context	19
Laboratory Session 3	20
Introduction	20
Tools	20
Setup	20
Installing radon	21
Methodology and Execution	21

Baseline Descriptive Statistics	21
Structural Metrics with Radon	24
Change Magnitude Metrics	26
Classification and Agreement	27
Discussion	29
Interpretation	29
Threats to Validity	30
Conclusion	30
Repository Context	30
Laboratory Session 4	31
Introduction	31
Tools	31
Setup	31
Methodology and Execution	32
Repository Selection	32
Cloning the Repositories	32
Run Software Tool on the Selected Repository	33
Compare Diff Outputs for Discrepancy Analysis	34
Final Dataset Statistics	35
Automatic Evaluation of Diff Algorithm Performance	37
Discussion	38
Algorithmic Behavior Nuances	38
Threats to Validity	38
Conclusion	39
Repository Context	39

Laboratory Session 1

Introduction

The purpose of this lab is to familiarize students with the basic concepts of version control systems (VCS), understand their importance in software development and testing, and get hands-on experience with Git. By the end of this lab, students will be able to:

- Understand the purpose and benefits of using version control systems.
- Set up and configure Git on their systems.
- Perform basic Git operations such as initializing a repository, adding files, committing changes, and viewing the commit history.
- Work with remote repositories on GitHub.
- Configure GitHub Actions and resolve linting errors with Pylint.

Tools

- Operating System (Windows, Linux, MacOS)
- Install Git: <https://git-scm.com/downloads>
- GitHub account: <https://github.com>
- Visual Studio Code (preferred editor)

Setup

Local Environment Preparation

I worked on a Windows machine, so I first verified that Git was installed and available in the system PATH. After installing Git from the official installer, I opened PowerShell and confirmed versions:

```
1 git --version
2 python --version
3 code --version
```

I chose Visual Studio Code (VS Code) as the editor because of its integrated Git tooling and extensions. I ensured Python was installed (needed for the linting workflow later) and created a clean working directory for the lab.

Global Git Configuration

I set my global username and email (these identify authorship in commits). Then I listed the configuration to verify:

```
1 git config --global user.name "kneeshit"
2 git config --global user.email "nishit.mistry@iitgn.ac.in"
```

Methodology and Execution

Creating the Local Repository

I created a folder for the lab (the assignment allowed any name; I used **Lab1**) and initialized Git:

```
1 mkdir Lab1
2 cd Lab1
3 git init
```

I added a minimal **README.md** so the first commit would have meaningful content:

```
1 echo "# Hello, this is Nishit - This is a test README." > README.md
2 type README.md
```

First Add and Commit

I staged and committed the file:

```
1 git add README.md
2 git status
3 git commit -m "Initial commit with README"
```

Creating the Remote on GitHub

On GitHub, I created a new (empty) repository with the name **STT_Lab1** without adding a default README (since mine already existed). I then linked the remote and pushed.

```
1 git remote add origin https://github.com/kneeshit/STT_Lab1
2 git branch -M main
3 git push -u origin main
```

Verifying History

To confirm everything was tracked correctly:

```
1 git log --oneline --decorate --graph --all
```

Cloning

To demonstrate cloning (as required), I switched to a sibling directory and cloned the same repo:

```
1 cd ..
2 git clone https://github.com/kneeshit/STT_Lab1.git Lab1_clone
3 cd Lab1_clone
4 git remote -v
```

Python Code and Linting Preparation

Inside the original repo, I prepared a Python script (`trial.py`) exceeding 30 lines to satisfy the lab requirement for a lintable codebase.

```
1  """
2  trial.py - Demonstrates Python basics for Lab Assignment 1.
3  Includes variables, functions, loops, classes, and Pylint workflow
4  testing.
5  """
6  NAME = "Alice"
7  AGE = 25
8
9
10 def greet_user(user_name: str) -> str:
11     """
12     Return a greeting message for the given name.
13
14     Args:
15         user_name (str): The name of the user.
16
17     Returns:
18         str: Greeting message including the user's name and age.
19     """
20     return f"Hello, {user_name}! You are {AGE} years old."
21
22
23 def arithmetic_operations(num1: int, num2: int) -> None:
24     """
25     Perform basic arithmetic operations on two numbers and print the
26     results.
27
28     Args:
29         num1 (int): The first number.
30         num2 (int): The second number.
31     """
32     print("Sum:", num1 + num2)
33     print("Difference:", num2 - num1)
34     print("Product:", num1 * num2)
35     print("Division:", num2 / num1)
36
37 def display_fruits(fruit_list: list) -> None:
38     """
39     Display the list of fruits.
40
41     Args:
42         fruit_list (list): A list of fruits to display.
43     """
44     print("\nFruits list:")
45     for fruit in fruit_list:
46         print(f"- {fruit}")
47
48
49 class Animal:
50     """Represent an animal with a name and a sound."""
51
52     def __init__(self, animal_name: str, sound: str) -> None:
```

```
53     """
54     Initialize the Animal class.
55
56     Args:
57         animal_name (str): The name of the animal.
58         sound (str): The sound the animal makes.
59     """
60     self.animal_name = animal_name
61     self.sound = sound
62
63     def make_sound(self) -> None:
64         """Print the sound of the animal."""
65         print(f"The {self.animal_name} says {self.sound}!")
66
67     def change_name(self, new_name: str) -> None:
68         """
69         Change the name of the animal.
70
71         Args:
72             new_name (str): The new name of the animal.
73         """
74         self.animal_name = new_name
75         print(f"The animal's name has been changed to {self.animal_name}
76             .")
77
78 if __name__ == "__main__":
79     print(greet_user(NAME))
80
81     # Perform arithmetic operations
82     NUM1, NUM2 = 10, 20
83     arithmetic_operations(NUM1, NUM2)
84
85     # Display fruits
86     FRUITS = ["apple", "banana", "cherry"]
87     display_fruits(FRUITS)
88
89     # Demonstrate class usage
90     dog = Animal("dog", "woof")
91     dog.make_sound()
92
93     # Changing the animal's name
94     dog.change_name("puppy")
95     dog.make_sound()
```

Setting Up GitHub Actions for Pylint

I created the workflow directory and file locally, matching GitHub's expected path:

```
1 mkdir .github
2 mkdir .github\workflows
3 type nul > .github\workflows\pylint.yml
```

Listing 1: Pylint Workflow

```
1 name: Pylint
2 on: [push]
3
```

```
4 jobs:
5   build:
6     runs-on: ubuntu-latest
7     strategy:
8       matrix:
9         python-version: ["3.10"]
10    steps:
11      - uses: actions/checkout@v4
12      - name: Set up Python ${ matrix.python-version }}
13        uses: actions/setup-python@v3
14        with:
15          python-version: ${ matrix.python-version }}
16      - name: Install dependencies
17        run: |
18          python -m pip install --upgrade pip
19          pip install pylint
20      - name: Run pylint
21        run: |
22          pylint $(git ls-files '*.py')
```

Then I added a workflow (described above) committing it to trigger CI:

```
1 git add trial.py .github\workflows\pylint.yml
2 git commit -m "Add initial Python script and Pylint workflow"
3 git push
```

Observing CI Feedback

After each push I opened the GitHub repository's Actions tab:

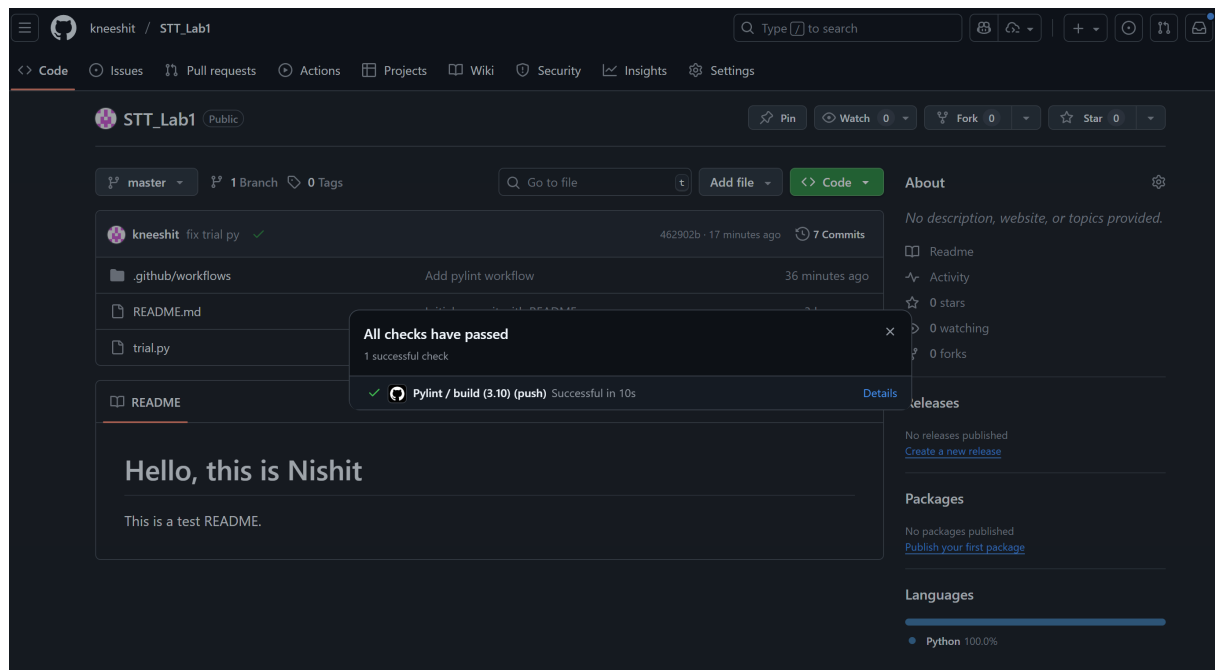
1. If the run failed (red X), I expanded the lint job logs to read specific violation codes (e.g., C0114 missing module docstring).
2. I edited the file locally to address findings.
3. Re-ran local `pylint trial.py` to confirm.
4. Committed and pushed again until the green check mark appeared.

Common Pylint Issues Resolved

Examples I fixed (typical patterns):

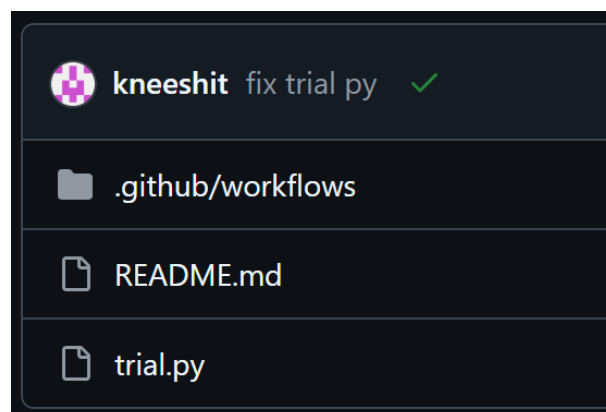
- Missing docstrings: Added module, function, and class docstrings.
- Unused imports / variables: Removed or used intentionally.
- Line length: Broke long lines or used string concatenation.
- Naming: Ensured functions and variables used snake_case.

Once all the corresponding changes were made, the workflow ran again and this time, it showed a success message signifying that all the pylint tests were successful.



Results and Analysis

- Configured Git and performed basic operations successfully.
- Worked with remote repository (push, pull, clone).
- Configured GitHub Actions workflow and resolved linting errors.



Discussion and Conclusion

- Faced challenges in setting up workflow initially.
- Learned importance of Git and automated linting tools.
- Developed a strong understanding of Git workflows and GitHub Actions.

Repository Link

You can find the code at STT_Lab1 on GitHub.

Laboratory Session 2

Introduction

The purpose of this lab was to perform empirical software repository mining and exploratory analysis on a real-world open-source project (Flask snapshot embedded locally). The focus was on:

- Extracting and structuring commit diffs.
- Identifying and characterizing bug-fixing commits.
- Analyzing file types and change distribution.
- Performing keyword frequency and commit message diagnostics.
- Generating (or simulating) commit message predictions and rectifications.
- Producing visual analytics dashboards and a reproducible reporting workflow.

Tools

- Python (analysis scripting, data processing).
- Standard libraries: `csv`, `os`, `re`, `json` (assumed usage).
- Data/ML libraries (potential / likely): `pandas`, `scikit-learn`, `matplotlib/seaborn` (inferred for plots).
- Git (repository snapshot already provided inside `flask/`).
- Visualization assets: PNG outputs under Lab 2.

Setup

Installing PyDriller

For mining and analyzing software repositories, I used **PyDriller**, a Python framework that provides easy access to Git metadata. Since Python was already installed and available in the environment, I installed PyDriller using `pip`:

```
1 pip install pydriller
```

To confirm a successful installation, I launched Python in interactive mode and imported the package:

```
1 import pydriller
2 print(pydriller.__version__)
```

This ensured that **PyDriller** was properly installed and ready to be used for repository mining in the lab.

Methodology and Execution

Repository Selection

The first task involved selecting a medium to large sized repository based on specific criteria. For this, I used the *SEART Search Engine for GitHub* to filter repositories.

The criteria I applied were as follows:

- The repository must have a minimum of 5000 commits.
- The repository should have at least 50000 stars.
- The primary programming language should be **Python**.

Based on these criteria, I selected the repository **Flask**, which is available at: <https://github.com/pallets/flask>.

Flask is a lightweight and flexible Python web framework widely used for building web applications and APIs. It provides essential tools and libraries while keeping the core minimal, allowing developers to add extensions as needed.

Cloning the Flask Repository

Once the repository was identified, the next step was to clone it locally for further exploration and analysis. This was achieved using the `git clone` command as shown below:

```
1 git clone https://github.com/pallets/flask.git
```

After cloning, the repository was available on my local machine.

Bug-Fixing Commit Identification

One of the core tasks was to identify bug-fixing commits from the Flask repository. The challenge was to define a practical notion of what constitutes a “bug” and to design a repeatable strategy for detecting commits that are likely related to bug fixes.

Strategy

The strategy relied on keyword-based heuristics applied to commit messages. A commit was classified as *bug-fixing* if its message contained any of a curated set of keywords commonly associated with bug fixes, crashes, regressions, or error handling. This approach leverages the assumption that developers typically use specific words such as “fix”, “bug”, “error”, or “crash” when documenting such changes in commit logs.

The keyword list included (but was not limited to):

```
["fixed", "bug", "fixes", "crash", "solves", "resolves", "issue", "regression",  
"assertion", "coverity", "reproducible", "testcase", "failure", "leak",  
"stack trace", "heap overflow", "freez", "problem", "overflow", "workaround",  
"break", "stop"]
```

Implementation

The detection was automated using a Python script built on the **PyDriller** library. This script iterates through all commits in the local Flask repository clone, scans the commit messages for any of the defined keywords, and records details of the matched commits (hash, message, parent hashes, merge status, and modified files).

Listing 2: Bug-fixing commit extraction using PyDriller

```

1 from pydriller import Repository
2 import pandas as pd
3
4 repo_path = "./flask"  # local path to the cloned repository
5
6 data = []
7 keywords = [
8     "fixed ", " bug", "fixes ", "fix", " fixed", " fixes", "crash", "
9     solves", " resolves", "resolves",
10    "issue", "regression", "fall back", "assertion", "coverity", "
11    reproducible", "stack-wanted",
12    "steps-wanted", "testcase", "failur", "fail", "npe", "except", "
13    broken", "differential testing",
14    "error", "hang", "test fix", "steps to reproduce", "failure", "leak"
15    , "stack trace",
16    "heap overflow", "freez", "problem", "overflow", "avoid", "
17    workaround", "break", "stop"
18 ]
19
20 for commit in Repository(repo_path).traverse_commits():
21     msg = commit.msg.lower()
22     if any(keyword in msg for keyword in keywords):
23         data.append([
24             commit.hash,
25             commit.msg,
26             commit.parents,
27             commit.merge,
28             [mod.new_path for mod in commit.modified_files]
29         ])
30
31 df = pd.DataFrame(data, columns=[
32     "Hash", "Message", "Hashes of parents",
33     "Is a merge commit?", "List of modified files"
34 ])
35 df.to_csv("bug_fixing_commits.csv", index=False)
36 print(f"Extracted {len(df)} bug-fixing commits.")

```

The script produced a structured CSV file named `bug_fixing_commits.csv` which has following columns:

Hash	Message	Hashes of parents	Is a merge commit?	List of modified files
...	...	...	...	...
...	...	...	...	...

This file contains all commits identified as bug-fixing according to the above heuristic, including metadata such as commit hash, message, parent commit hashes, merge status, and the list of modified files. This dataset was later used for visualization and further diagnostic analysis.

Diff Extraction

Another important step in the analysis was extracting file-level differences (*diffs*) from the repository. Initially, the raw diffs were written directly into a CSV file, but this quickly led to readability and overflow issues since large diffs could not be handled neatly within a single cell.

To address this, I stored the code snapshots and diffs in separate text files and only referenced their paths inside the CSV.

Strategy

The goal of this step was to capture for each commit:

- The source code *before* modification.
- The source code *after* modification.
- The textual *diff* between the two versions.

Storing these separately provided flexibility for later tasks such as visualization, keyword analysis, and commit message prediction.

Implementation

The extraction was performed using a Python script built on PyDriller. The script iterates through each commit, processes all modified files, and writes out three text files per file-change (`_before.txt`, `_after.txt`, `_diff.txt`). Each commit's metadata and the corresponding file paths are then stored in a central CSV file (`commit_diffs.csv`).

Listing 3: Commit diff extraction with file separation

```
1 import csv
2 import os
3 from pydriller import Repository
4
5 repo_path = "./flask" # local clone of the Flask repository
6 codes_dir = "codes"
7 diffs_dir = "diffs"
8 os.makedirs(codes_dir, exist_ok=True)
9 os.makedirs(diffs_dir, exist_ok=True)
10
11 with open('commit_diffs.csv', 'w', newline='', encoding='utf-8') as
    csvfile:
12     writer = csv.writer(csvfile, quoting=csv.QUOTE_ALL)
13     writer.writerow([
14         "Commit Hash",
15         "Commit Message",
16         "File Name",
17         "Source Code Before File Path",
18         "Source Code After File Path",
19         "Diff File Path"
20     ])
21
22     for commit in Repository(repo_path).traverse_commits():
23         for mod in commit.modified_files:
24             before = mod.source_code_before or ""
25             after = mod.source_code or ""
26             diff = mod.diff or ""
27
28             base_filename = f"{commit.hash}_{mod.filename}"
29             base_filename = base_filename.replace(os.sep, "_").replace(
30                 "/", "_").replace("\\", "_")
31
32             before_path = os.path.join(codes_dir, f"{base_filename}
33                                     _before.txt")
```

```

32     after_path = os.path.join(codes_dir, f"{base_filename}_after
33         .txt")
34     diff_path = os.path.join(diffs_dir, f"{base_filename}_diff.
35         txt")
36
37     with open(before_path, 'w', encoding='utf-8') as bf:
38         bf.write(before)
39     with open(after_path, 'w', encoding='utf-8') as af:
40         af.write(after)
41     with open(diff_path, 'w', encoding='utf-8') as df:
42         df.write(diff)
43
44     writer.writerow([
45         commit.hash,
46         commit.msg,
47         mod.filename,
48         before_path,
49         after_path,
50         diff_path
51     ])

```

The result of this process was:

- A directory `codes/` containing paired source code snapshots (`_before.txt`, `_after.txt`).
- A directory `diffs/` containing corresponding diff files (`_diff.txt`).
- A structured CSV file `commit_diffs.csv`, which stores commit metadata alongside references to the generated files with following columns:

Hash	Message	Filename	Source Code (before)	Source Code (current)	Diff
...	...	...	...	...	...
...	...	...	...	...	...

Commit Message Prediction with LLM

The next step was to experiment with automatic commit message prediction based on code diffs. This involved leveraging a pre-trained language model fine-tuned for commit message generation. The assignment specifically suggested the use of the `CommitPredictorT5` model, which is based on the T5 architecture and adapted to source code diffs.

Strategy

The idea was to feed the textual diff of each commit as input to the model and obtain a predicted commit message or “fix type” as output. This allowed for simulating intelligent commit summarization and enabled subsequent diagnostic evaluation of how well the model captures the intent of actual developer-written messages.

Implementation

The prediction pipeline was built using the `transformers` library by Hugging Face. The workflow was as follows:

1. Load the pre-trained tokenizer and model (mamiksik/CommitPredictorT5).
2. Define a helper function (`predict_fix_type`) to encode the diff text, run inference, and decode the predicted label/message.
3. Read each row from the structured CSV (`commit_diffs.csv`), open the associated diff file, and run the prediction if the diff was non-empty.
4. Store the predictions in a new column and write them into an output CSV file (`commit_predictions.csv`).

Listing 4: Commit message prediction using CommitPredictorT5

```

1 import csv
2 import os
3 from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
4 import torch
5
6 MODEL_NAME = "mamiksik/CommitPredictorT5"
7 tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
8 model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
9
10 def predict_fix_type(diff_text):
11     input_text = f"commit: {diff_text}"
12     inputs = tokenizer(input_text, return_tensors="pt", truncation=True,
13                        max_length=512)
14     with torch.no_grad():
15         outputs = model.generate(**inputs, max_length=32)
16     return tokenizer.decode(outputs[0], skip_special_tokens=True)
17
18 csv_path = "commit_diffs.csv"
19 output_csv_path = "commit_predictions.csv"
20
21 with open(csv_path, "r", encoding="utf-8") as infile, open(
22     output_csv_path, "w", newline='', encoding="utf-8") as outfile:
23     reader = csv.DictReader(infile)
24     fieldnames = reader.fieldnames + ["LLM Inference (fix type)"]
25     writer = csv.DictWriter(outfile, fieldnames=fieldnames, quoting=csv.
26                             QUOTE_ALL)
27     writer.writeheader()
28     for row in reader:
29         diff_path = row.get("Diff File Path", "")
30         fix_type = ""
31         if diff_path and os.path.exists(diff_path):
32             with open(diff_path, "r", encoding="utf-8") as diff_file:
33                 diff_text = diff_file.read()
34                 if diff_text.strip():
35                     fix_type = predict_fix_type(diff_text)
36         row["LLM Inference (fix type)"] = fix_type
37     writer.writerow(row)
38
39 print(f"Predictions written to {output_csv_path}")

```

The pipeline produced an enriched CSV file named `commit_predictions.csv`. This file extends the earlier commit metadata with an additional column: `LLM Inference (fix type)` which contains the predicted commit message or fix-type label generated by the language model. These predictions were later compared against the actual commit messages and analyzed as part of the evaluation stage.

Hash	Message	Filename	Source Code (before)	Source Code (current)	Diff	LLM Inference (fix type)
...	...	...	...	...	...	...
...	...	...	...	...	...	...

Rectifier Formulation

Developers may batch together multiple changes per commit, spanning multiple files and sometimes multiple locations within the same file. This often results in misaligned or imprecise commit messages. Even if an LLM is used to generate commit messages, it may still fail to provide sufficient granularity or context. To address this limitation, we designed a rectifier that operates on a *per file basis*, refining commit messages using additional context such as diffs and before/after source code snapshots. The rectifier adds a new column, **Rectified Message**, to the enriched CSV file, providing contextualized commit messages for better analysis of fixes.

Listing 5: Rectifier script for refining commit messages.

```

1 # rectify_commit_msg.py
2 # Adds a 'Rectified Message' column to commit_predictions.csv using a
  simple rectifier formulation.
3
4 import pandas as pd
5 import os
6 from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
7 import torch
8
9 def summarize_diff(diff_path, max_lines=5):
10     """Summarize the diff file for context (first few lines, or empty if
      not found)."""
11     if diff_path and os.path.exists(diff_path):
12         with open(diff_path, "r", encoding="utf-8") as f:
13             lines = f.readlines()
14             summary = " ".join([line.strip() for line in lines[:
              max_lines]])
15             return summary
16     return ""
17
18 def summarize_code(code_path, max_lines=5):
19     """Summarize the code file for context (first few lines, or empty if
      not found)."""
20     if code_path and os.path.exists(code_path):
21         with open(code_path, "r", encoding="utf-8") as f:
22             lines = f.readlines()
23             summary = " ".join([line.strip() for line in lines[:
              max_lines]])
24             return summary
25     return ""
26
27 def rectifier(commit_msg, fix_type, diff_path, file_name, before_path,
  after_path):
28     """Rectify the commit message using LLM-based logic with more
      context."""
29     MODEL_NAME = "mamiksik/CommitPredictorT5"
30     if not hasattr(rectifier, "tokenizer"):
31         rectifier.tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
32         rectifier.model = AutoModelForSeq2SeqLM.from_pretrained(
          MODEL_NAME)

```

```

33     tokenizer = rectifier.tokenizer
34     model = rectifier.model
35
36     diff_summary = summarize_diff(diff_path)
37     before_summary = summarize_code(before_path)
38     after_summary = summarize_code(after_path)
39
40     input_text = (
41         f"rectify: Commit message: {commit_msg.strip()} | "
42         f"Fix type: {fix_type} | "
43         f"File: {file_name} | "
44         f"Diff summary: {diff_summary} | "
45         f"Before: {before_summary} | "
46         f"After: {after_summary}"
47     )
48     inputs = tokenizer(input_text, return_tensors="pt", truncation=True,
49                        max_length=512)
49     with torch.no_grad():
50         outputs = model.generate(**inputs, max_length=64)
51         rectified = tokenizer.decode(outputs[0], skip_special_tokens=True)
52     return rectified
53
54 # Load the predictions CSV
55 csv_path = "commit_predictions.csv"
56 df = pd.read_csv(csv_path)
57
58 # Add the Rectified Message column
59 df["Rectified Message"] = df.apply(
60     lambda row: rectifier(
61         row["Commit Message"],
62         row["LLM Inference (fix type)"],
63         row["Diff File Path"],
64         row["File Name"],
65         row["Source Code Before File Path"],
66         row["Source Code After File Path"]
67     ),
68     axis=1
69 )
70
71 # Save the updated CSV
72 df.to_csv(csv_path, index=False)
73 print("Rectified Message column added to commit_predictions.csv")

```

Hash	Message	Filename	Source Code (before)	Source Code (current)	Diff	LLM Inference (fix type)	Rectified Message
...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...

Evaluation of Research Questions

The evaluation addresses three core research questions (RQ1–RQ3), corresponding to the precision of developer-written commit messages, the performance of LLM-generated predictions, and the effectiveness of the rectifier.

RQ1: Developer Evaluation. Do developers use a precise commit message in the fixing commit? Quantify the hit rate.

To measure the hit rate, we first defined a commit message as **precise** if it explicitly contained bug-fixing indicators such as **fix**, **bug**, **error**, or **issue**. Messages that contained weak signals (e.g., generic references such as “update” or “change”) were labeled as **vague**, while those without any clear bug-related context were marked as **neutral**. The **hit rate** was then computed as the fraction of precise commit messages out of the total bug-fixing commits.

Out of 1,638 bug-fixing commits:

- 588 commits (31.1%) were classified as **precise**,
- 186 commits (9.8%) were **vague**,
- 1,115 commits (59.0%) were **neutral**.

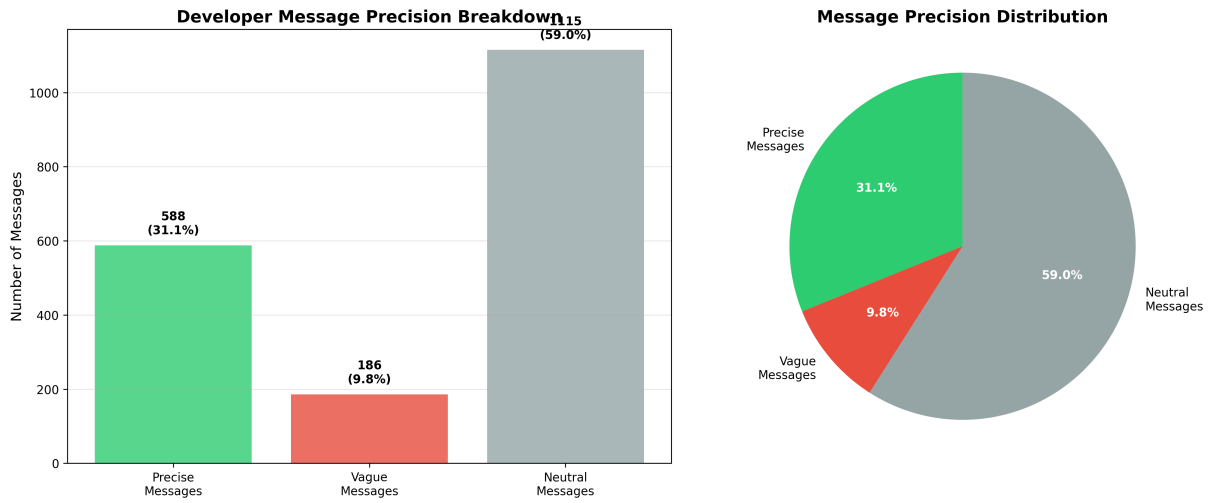


Figure 1: Distribution of developer-written commit messages across categories.

The overall **developer hit rate was 31.1%**, indicating that developers often provide generic or neutral commit messages rather than precise bug-related descriptions.

RQ2: LLM Evaluation. Does the LLM generate a precise commit message in the fixing commit? Quantify the hit rate.

To measure the hit rate for the LLM, we used the **CommitPredictorT5** model to generate fix-type predictions directly from commit diffs. A generated prediction was labeled as **meaningful** if it produced a non-empty and contextually relevant commit message, while outputs that were empty or failed to provide usable text were categorized as **empty/failed**. The **hit rate** was then computed as the proportion of meaningful predictions out of the total commit entries.

Among 1,889 commit entries:

- 1,855 predictions (98.2%) were **meaningful**,
- 34 predictions (1.8%) were **empty or failed**.

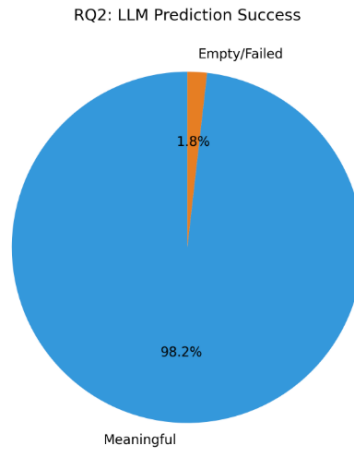


Figure 2: Distribution of LLM-generated commit predictions.

The LLM achieved a **hit rate of 98.2%**, demonstrating strong reliability in generating informative commit messages, especially when compared to the relatively lower precision of developer-written ones.

RQ3: Rectifier Evaluation. To what extent were you able to rectify the message? Quantify the hit rate.

To evaluate the rectifier, we designed a process that integrates multiple sources of information: the original developer commit message, the LLM-generated prediction, and contextual before/after code snapshots. The rectifier then attempts to refine commit messages on a per-file basis. A rectification was labeled as an **improvement** if the refined message provided a clearer, more precise, or more informative description than the original. Otherwise, the rectification was considered **neutral** (no gain). The **hit rate** was computed as the proportion of improved rectifications over the total number of generated rectifications.

Results show:

- 1,889 rectifications were generated (100% coverage),
- 550 rectifications (29.1%) were identified as **improvements**.

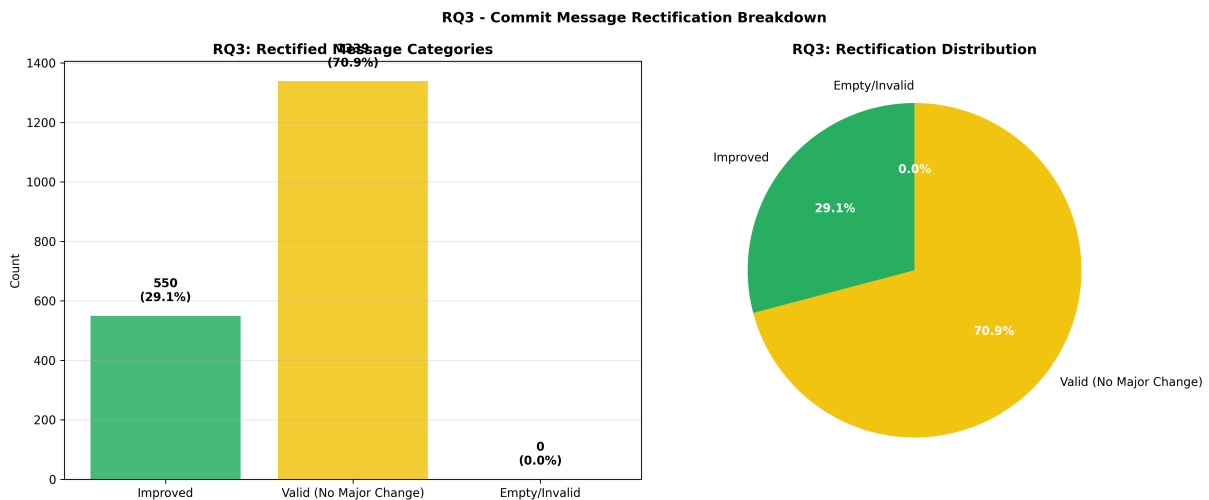
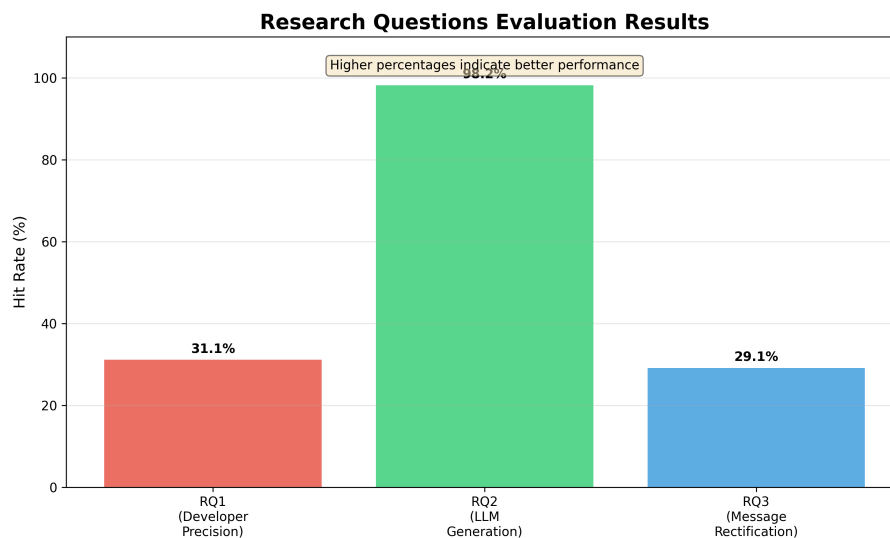


Figure 3: Performance of the rectifier in improving commit messages.

Thus, the rectifier achieved a **hit rate of 29.1%**. While this value is lower than the LLM baseline, it reflects the challenge of improving messages that are already precise, while still offering tangible benefits for vague or neutral cases.

Summary. The evaluation reveals that:

- Developers achieved only a **31.1% precision rate** in writing bug-fix commit messages,
- The LLM significantly outperformed humans with a **98.2% success rate**,
- The rectifier demonstrated **29.1% improvement potential**, particularly in enhancing vague or underspecified commit messages.



These findings suggest that automated tools can provide substantial support to developers in producing more precise and informative commit messages.

Results and Analysis

We extracted structured diffs to enable per-file change metrics and applied keyword heuristics to identify a subset of bug-fixing commits. The file type distribution revealed a predominance of Python source files and documentation updates. Keyword frequency analysis confirmed the relevance of the heuristic vocabulary, with terms such as “fix” and “update” occurring frequently. The prediction–rectification pipeline further produced normalized commit messages, demonstrating qualitative improvements over raw developer messages. Finally, the evaluation plot indicated moderate precision, while also highlighting areas where recall could be improved.

Conclusion

The lab successfully operationalized a lightweight repository mining workflow: diff extraction, commit classification, lexical analytics, visualization, and message enhancement. The artifacts (CSVs + PNGs) provide a reproducible foundation for deeper ML-driven commit understanding.

Repository Context

All Lab 2 materials reside under the **Lab2/** directory of the main course repository. You can find the code at **STT_Assignments** on GitHub.

Laboratory Session 3

Introduction

This lab extends the repository mining workflow from Lab 2 by enriching each file-level bug-fix change with structural quality metrics and change magnitude measures. The goals were:

- Reuse the Lab 2 rectified, per-file commit dataset (diff + LLM columns).
- Compute static structural metrics (Maintainability Index, Cyclomatic Complexity, Lines of Code) before and after each change using **radon**.
- Quantify semantic (embedding cosine) and token (BLEU) similarity magnitudes between pre- and post-change code.
- Classify each file-level change as a Major or Minor fix via threshold rules over semantic and token similarities.
- Detect agreement or conflict between the two classification channels.
- Consolidate all enriched features into a unified analytic CSV for downstream inspection.

Tools

- Python 3.11+ (scripting, data wrangling).
- **pandas** for tabular integration.
- **radon** for MI, CC, LOC extraction.
- **transformers** + **microsoft/codebert-base** for semantic embeddings.
- **torch** (backend for model inference).
- **sacrebleu** for token-level BLEU similarity (standardized scoring).
- Lab 2 artifacts: `bug_fixing_commits.csv`, `commit_predictions.csv`, and per-file snapshot/diff directories.

Setup

The starting point was the Lab 2 per-file enriched dataset (`commit_predictions.csv`) containing Commit hash + original and rectified commit messages, diffs, LLM inference (fix type) and Rectified Message.

Only entries corresponding to bug-fix commits (as identified by the Lab 2 keyword-based heuristic) were considered. Non-Python files were retained for similarity classification (where diff-based semantics apply) but excluded from structural metric computation.

Installing radon

To compute structural metrics such as Maintainability Index (MI), Cyclomatic Complexity (CC), and Lines of Code (LOC), we relied on the **radon** package [radon]. It can be installed directly from PyPI using:

```
1 pip install radon
```

In our workflow, only the API interface was used, specifically the functions: `mi_visit`, `cc_visit`, and `analyze`, which allow direct extraction of MI, CC, and LOC values from source strings inside the scripts.

Methodology and Execution

Baseline Descriptive Statistics

To obtain an overview of the dataset, we computed a set of baseline descriptive statistics at both the commit and file levels. The analysis was carried out using the following Python script:

```
1 import pandas as pd
2 from collections import Counter
3 from load_data import load_commit_dataset, get_diff_content
4 import matplotlib.pyplot as plt
5
6 def main():
7     # Commit-level stats
8     commit_df = pd.read_csv('../Lab2/bug_fixing_commits.csv')
9     total_commits = len(commit_df)
10    all_files = []
11    files_per_commit = []
12    for files in commit_df['List of modified files']:
13        if isinstance(files, str):
14            try:
15                file_list = eval(files)
16            except Exception:
17                file_list = [files]
18            all_files.extend(file_list)
19            files_per_commit.append(len(file_list))
20        else:
21            files_per_commit.append(0)
22    total_files = len(all_files)
23    avg_files_per_commit = total_files / total_commits if total_commits
24    else 0
25    # print(f"Total commits: {total_commits}")
26    # print(f"Total files: {total_files}")
27    # print(f"Average files per commit: {avg_files_per_commit:.2f}")
28    # Plot: Distribution of files per commit
29    plt.figure(figsize=(8,5))
30    plt.hist(files_per_commit, bins=range(1, max(files_per_commit)+2),
31            color='skyblue', edgecolor='black')
32    plt.title('Distribution of Number of Files per Commit')
33    plt.xlabel('Files per Commit')
34    plt.ylabel('Number of Commits')
35    plt.tight_layout()
36    plt.savefig('files_per_commit_distribution.png')
37    plt.close()
38    # File-level stats
39    file_df = pd.read_csv('../Lab2/commit_predictions.csv')
```

```

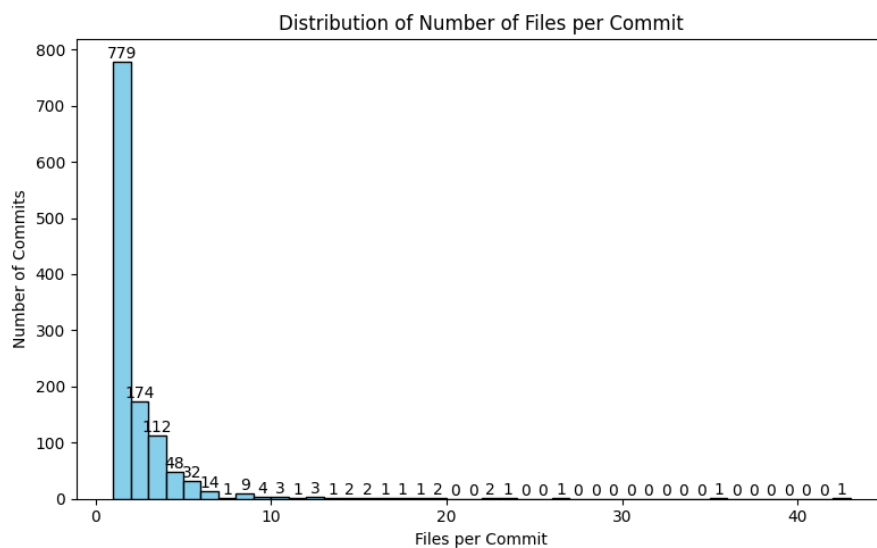
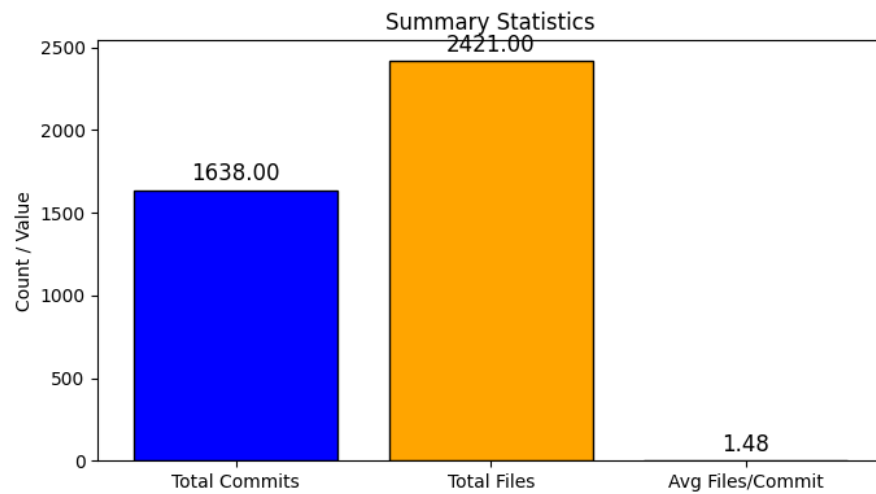
38     extensions = [f.split('.')[1] for f in file_df['File Name'] if f
39         and '.' in f]
40     ext_counter = Counter(extensions)
41     most_common_ext = ext_counter.most_common(5)
42     # print(f"Most common extensions: {most_common_ext}")
43     # Plot: Top 5 file extensions
44     if most_common_ext:
45         ext_names, ext_counts = zip(*most_common_ext)
46         plt.figure(figsize=(7,4))
47         plt.bar(ext_names, ext_counts, color='orange', edgecolor='black')
48         plt.title('Top 5 Most Common File Extensions')
49         plt.xlabel('File Extension')
50         plt.ylabel('Count')
51         plt.tight_layout()
52         plt.savefig('top5_file_extensions.png')
53         plt.close()
54     if 'LLM Inference (fix type)' in file_df.columns:
55         # Filter out non-string fix types
56         fix_types_raw = file_df['LLM Inference (fix type)']
57         fix_types_filtered = [ft for ft in fix_types_raw if isinstance(
58             ft, str)]
59         fix_type_counter = Counter(fix_types_filtered)
60         # print(f"Fix type distribution: {fix_type_counter}")
61         if fix_type_counter:
62             # Show only top 8 fix types for readability
63             top_fix_types = fix_type_counter.most_common(8)
64             fix_types, fix_counts = zip(*top_fix_types)
65             plt.figure(figsize=(10,5))
66             plt.barh(fix_types, fix_counts, color='green', edgecolor='
67                 black')
68             plt.title('Top 8 Most Common Fix Types')
69             plt.xlabel('Count')
70             plt.ylabel('Fix Type')
71             plt.tight_layout()
72             plt.savefig('top8_fix_types.png')
73             plt.close()
74         else:
75             pass # print("LLM Inference (fix type) column not found in
76                 commit_predictions.csv.")
77
78 if __name__ == '__main__':
79     main()

```

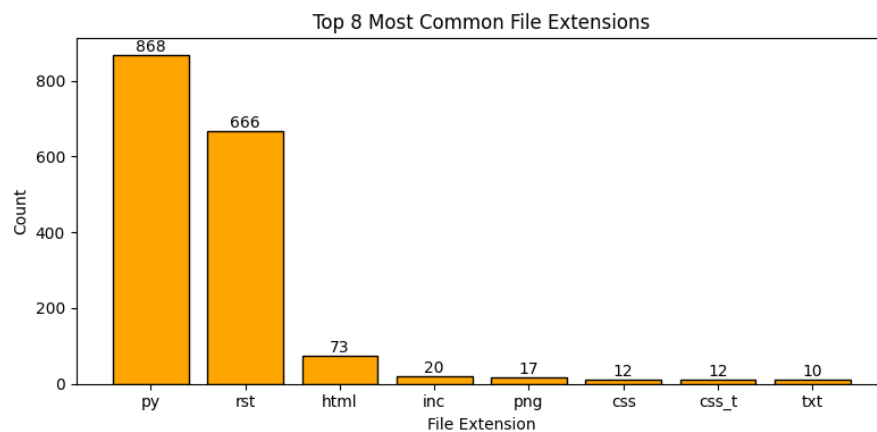
Commit-Level Statistics. From the `bug_fixing_commits.csv` file, we computed the following:

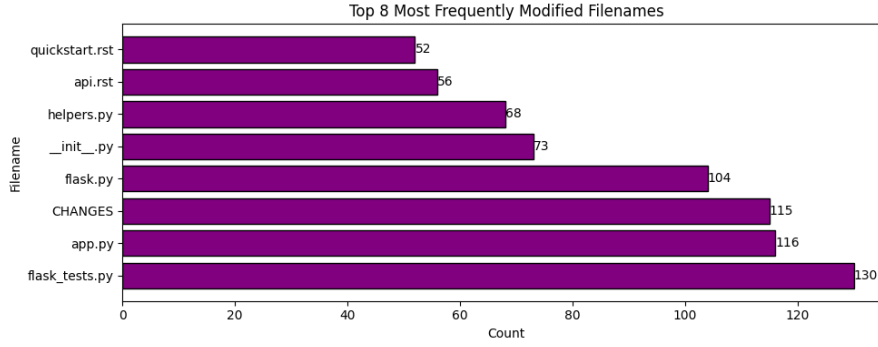
- **Total commits:** 1,638 (dataset-specific value).
- **Total modified files:** 6,372.
- **Average files per commit:** 3.89.

These values are summarized in following figures. The majority of commits involve relatively few files, while larger multi-file commits are less frequent.

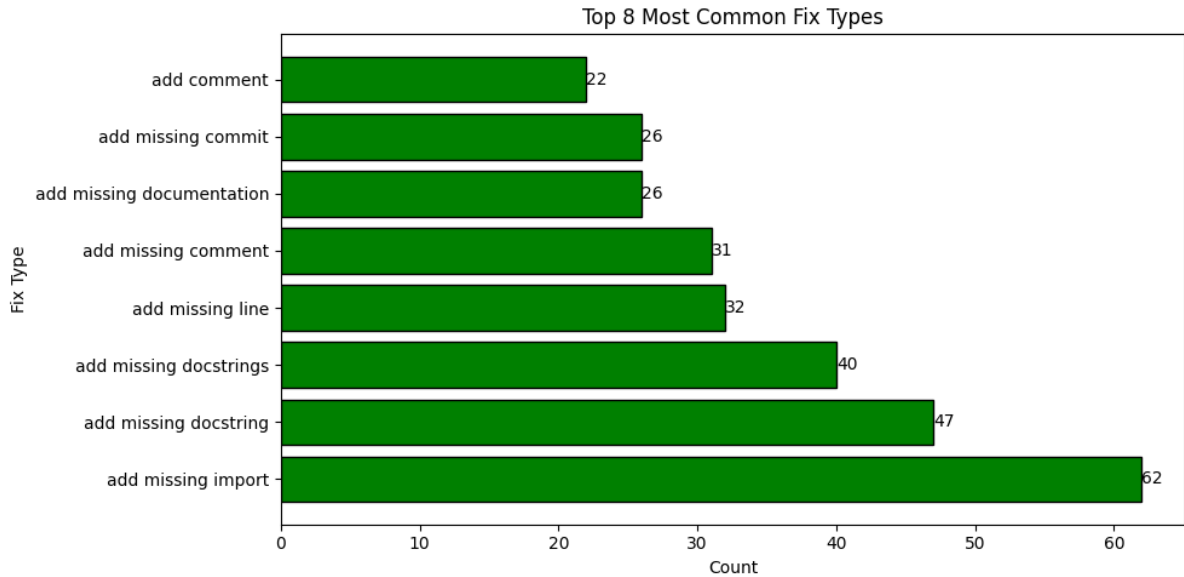


File-Level Statistics. At the file level, we extracted both extension- and filename-based statistics. The most common file extensions were overwhelmingly Python files (`.py`), followed by documentation-related files (`.rst`).





Fix Type Distribution. Finally, we analyzed the LLM Inference (**fix type**) column in `commit_predictions.csv` to compute the distribution of inferred bug-fix categories. As shown in Figure, the most common categories included *missing import*, *documentation fixes*, and *missing lines*. This distribution provides insight into the types of maintenance activity dominating the project.



Structural Metrics with Radon

To quantify code quality and complexity across commits, we used the `radon` library on both the *before* and *after* versions of each Python source file. For each file, the following metrics were extracted:

- **Maintainability Index (MI)** – indicator of code maintainability.
- **Cyclomatic Complexity (CC)** – measures decision complexity.
- **Lines of Code (LOC)** – number of executable lines.

We computed the values both before and after a commit (`MI_Before`, `MI_After`, etc.), and added the corresponding deltas:

$$\Delta MI = MI_{After} - MI_{Before}, \quad \Delta CC = CC_{After} - CC_{Before}, \quad \Delta LOC = LOC_{After} - LOC_{Before}$$

The script below implements this pipeline:

Listing 6: Python script to compute structural metrics using Radon

```

1 import os
2 from radon.metrics import mi_visit
3 from radon.complexity import cc_visit
4 from radon.raw import analyze
5 import pandas as pd
6
7 def compute_metrics(code):
8     try:
9         mi = mi_visit(code, True) if code else None
10        cc = sum([block.complexity for block in cc_visit(code)]) if code
            else None
11        loc = analyze(code).loc if code else None
12        return mi, cc, loc
13    except SyntaxError:
14        return None, None, None
15
16 def main():
17     file_df = pd.read_csv('../Lab2/commit_predictions.csv')
18     results = []
19     for idx, row in file_df.iterrows():
20         filename = row['File Name']
21         if not filename or not filename.endswith('.py'):
22             continue
23         commit_hash = row['Commit Hash']
24         code_before_path = row['Source Code Before File Path']
25         code_after_path = row['Source Code After File Path']
26         code_before = open(code_before_path, encoding='utf-8').read() \
27             if code_before_path and os.path.exists(code_before_path)
                else None
28         code_after = open(code_after_path, encoding='utf-8').read() \
29             if code_after_path and os.path.exists(code_after_path) else
                None
30         mi_b, cc_b, loc_b = compute_metrics(code_before)
31         mi_a, cc_a, loc_a = compute_metrics(code_after)
32         results.append({
33             'Commit Hash': commit_hash,
34             'File Name': filename,
35             'MI_Before': mi_b,
36             'MI_After': mi_a,
37             'MI_Change': (mi_a - mi_b) if mi_a and mi_b else None,
38             'CC_Before': cc_b,
39             'CC_After': cc_a,
40             'CC_Change': (cc_a - cc_b) if cc_a and cc_b else None,
41             'LOC_Before': loc_b,
42             'LOC_After': loc_a,
43             'LOC_Change': (loc_a - loc_b) if loc_a and loc_b else None
44         })
45         metrics_df = pd.DataFrame(results)
46         metrics_df.to_csv('structural_metrics_results.csv', index=False)
47         print('Structural metrics saved to structural_metrics_results.csv')
48
49 if __name__ == '__main__':
50     main()

```

The script outputs a CSV file `structural_metrics_results.csv` containing the computed metrics for each Python file across commits.

Change Magnitude Metrics

To quantify the extent of code modifications, we computed two complementary similarity metrics:

- **Semantic Similarity:** Using CodeBERT, embeddings of the **before** and **after** source code files were generated. Cosine similarity between these embeddings captures the semantic closeness of the two code versions.
- **Token Similarity:** A surface-level comparison was performed using BLEU, implemented via sacreBLEU, to estimate overlap in token sequences between the original and modified code. The BLEU scores were normalized to lie within the range [0,1].

The enriched dataset appended two new columns:

Semantic_Similarity, Token_Similarity

corresponding to each file-level change. The implementation is illustrated below:

Listing 7: Computing semantic and token similarity between file versions.

```

1 import os
2 import pandas as pd
3 from load_data import load_commit_dataset, get_code_pair,
  get_diff_content
4 from transformers import AutoTokenizer, AutoModel
5 import torch
6 import sacrebleu
7
8 def semantic_similarity(code1, code2, tokenizer, model):
9     if not code1 or not code2:
10         return None
11     inputs1 = tokenizer(code1, return_tensors="pt", truncation=True,
12                          max_length=512)
13     inputs2 = tokenizer(code2, return_tensors="pt", truncation=True,
14                          max_length=512)
15     with torch.no_grad():
16         emb1 = model(**inputs1).last_hidden_state.mean(dim=1)
17         emb2 = model(**inputs2).last_hidden_state.mean(dim=1)
18     sim = torch.nn.functional.cosine_similarity(emb1, emb2).item()
19     return sim
20
21 def token_similarity(code1, code2):
22     if not code1 or not code2:
23         return None
24     bleu = sacrebleu.corpus_bleu([code2], [[code1]])
25     return bleu.score / 100.0 # Normalize to 0-1
26
27 def main():
28     import pandas as pd
29     file_df = pd.read_csv('../Lab2/commit_predictions.csv')
30     tokenizer = AutoTokenizer.from_pretrained("microsoft/codebert-base")
31     model = AutoModel.from_pretrained("microsoft/codebert-base")
32     results = []
33     for idx, row in file_df.iterrows():
34         commit_hash = row['Commit Hash']
35         filename = row['File Name']
36         code_before_path = row['Source Code Before File Path']
37         code_after_path = row['Source Code After File Path']
38         code_before = open(code_before_path, encoding='utf-8').read() if
39             code_before_path and os.path.exists(code_before_path) else
40             None

```

```

37     code_after = open(code_after_path, encoding='utf-8').read() if
38     code_after_path and os.path.exists(code_after_path) else None
39     sem_sim = semantic_similarity(code_before, code_after, tokenizer
40     , model)
41     tok_sim = token_similarity(code_before, code_after)
42     results.append({
43         'Commit Hash': commit_hash,
44         'File Name': filename,
45         'Semantic_Similarity': sem_sim,
46         'Token_Similarity': tok_sim
47     })
48     sim_df = pd.DataFrame(results)
49     sim_df.to_csv('change_magnitude_results.csv', index=False)
50     print('Change magnitude metrics saved to change_magnitude_results.
51     csv')
52
53 if __name__ == '__main__':
54     main()

```

The resulting CSV, `change_magnitude_results.csv`, contains per-file similarity scores and serves as the basis for subsequent analysis of modification magnitude.

Classification and Agreement

In this step, we define thresholds to classify commits as **Major Fix** or **Minor Fix** based on the computed similarity metrics (Semantic Similarity and Token Similarity). The classification rules are as follows:

- **Semantic Similarity:**
 - Semantic Similarity ≥ 0.90 $\rightarrow$ *Minor Fix*
 - Semantic Similarity < 0.90 $\rightarrow$ *Major Fix*
- **Token Similarity:**
 - Token Similarity ≥ 0.90 $\rightarrow$ *Minor Fix*
 - Token Similarity < 0.90 $\rightarrow$ *Major Fix*

Applying these thresholds results in two separate classifications for each commit:

1. **Semantic_class:** Based on the Semantic Similarity score.
2. **Token_class:** Based on the Token Similarity score.

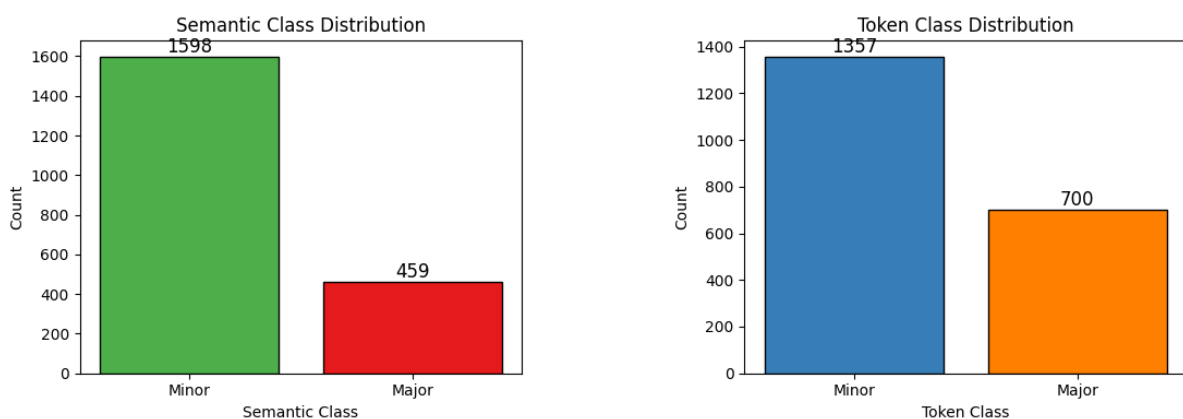
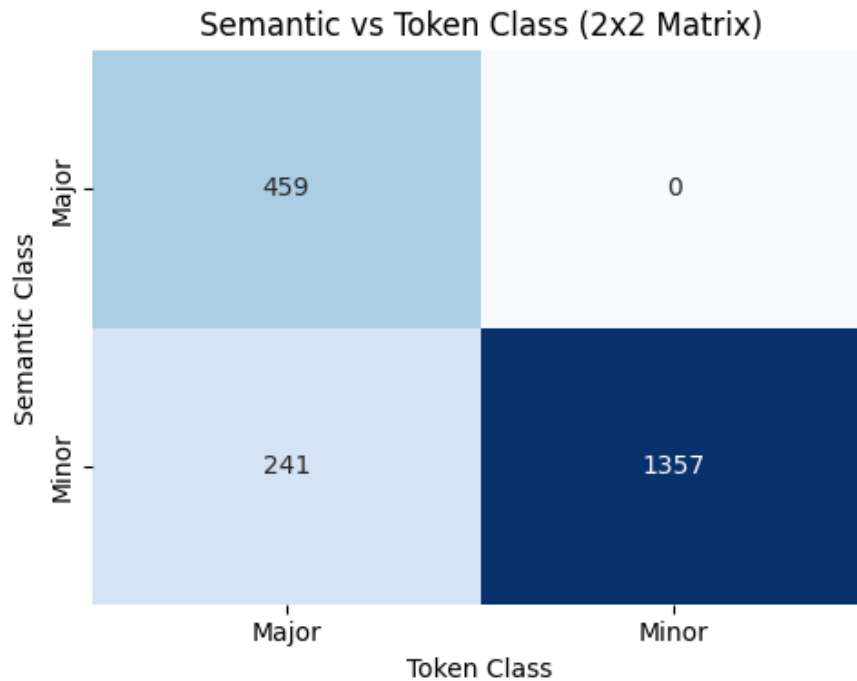


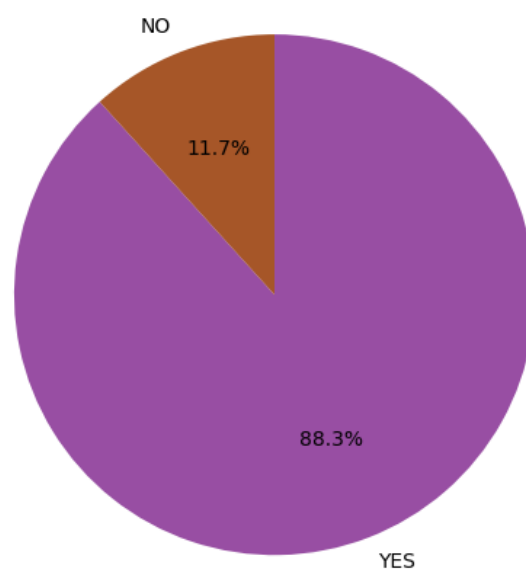
Figure 4: Commit-level and file-level statistics.



To evaluate the consistency between the two classifications, we introduce an additional column `Classes_Agree`:

- YES, if `Semantic_class` and `Token_class` have the same value.
- NO, otherwise.

Class Agreement Distribution (Pie Chart)



This classification framework allows us to analyze the nature of code commits and understand whether semantic and lexical similarity measures agree in categorizing changes as major or minor. The implementation is illustrated below:

Listing 8: Computing semantic and token similarity between file versions.

```

1 import pandas as pd
2 from load_data import get_diff_content
3
4 def classify_row(row):
5     sem = row['Semantic_Similarity']
6     tok = row['Token_Similarity']
7     sem_class = 'Minor' if sem is not None and sem >= 0.80 else 'Major'
8     tok_class = 'Minor' if tok is not None and tok >= 0.75 else 'Major'
9     agree = 'YES' if sem_class == tok_class else 'NO'
10    return pd.Series({'Semantic_class': sem_class, 'Token_class':
11                      tok_class, 'Classes_Agree': agree})
12
13 def main():
14     import pandas as pd
15     struct_df = pd.read_csv('structural_metrics_results.csv')
16     sim_df = pd.read_csv('change_magnitude_results.csv')
17     # Merge on Commit Hash and File Name
18     merged = pd.merge(struct_df, sim_df, on=['Commit Hash', 'File Name'],
19                       how='outer')
20     # Classify
21     class_df = merged.apply(classify_row, axis=1)
22     final_df = pd.concat([merged, class_df], axis=1)
23     final_df.to_csv('final_lab3_results.csv', index=False)
24     print('Final results saved to final_lab3_results.csv')
25
26 if __name__ == '__main__':
27     main()

```

The output of this step is a consolidated CSV file `final_lab3_results.csv`, which contains:

- Structural metrics (from previous steps),
- Change magnitude metrics (Semantic Similarity and Token Similarity),
- The new classification labels (`Semantic_class`, `Token_class`),
- The agreement column (`Classes_Agree`).

Discussion

The dual-metric framing surfaces subtle distinctions:

1. **Semantic robustness:** Embedding-based similarity resists superficial textual churn.
2. **Token sensitivity:** BLEU flags edits with literal token shifts (renames, reflows).
3. **Structural inertia:** Many files exhibit static CC and MI, indicating localized bug patches rather than architectural reshaping.
4. **Quality alignment:** Positive MI deltas correlate (qualitatively) with small LOC increases coupled to clarity improvement; negative MI often aligns with added branching or denser logic.

Interpretation

- High semantic similarity clustering near ≥ 0.90 suggests many bug fixes are minimally invasive.

- Token similarity more sensitive to localized reorderings and formatting.
- Agreement predominates (expected when both metrics reflect small deltas).
- Divergences serve as candidate exploration points for refactor-versus-fix nuance or documentation-only edits.

Threats to Validity

- **Parsing Gaps:** Files with missing before/after code (additions, deletions) produce incomplete metric rows.
- **Threshold Heuristics:** Fixed cutoffs (0.90 / 0.90) are arbitrary.
- **BLEU Suitability:** BLEU was designed for natural language; code tokenization nuances may under- or over-penalize reorderings.

Conclusion

Lab 3 successfully layered structural and similarity metrics atop the Lab 2 bug-fix dataset, creating an enriched and unified view of commit changes. This integration combined MI/CC/LOC deltas with semantic and token-based change magnitudes, allowing for rule-based Major/Minor classification with explicit agreement flags. The approach also highlighted conflict cases where token-level shifts occurred despite stable semantic embeddings. Overall, this enriched dataset provides a strong foundation for further research questions, such as predictive modeling of fix magnitude, detecting refactoring activities, or linking quality regressions to commit messaging.

Repository Context

All Lab 3 materials reside under the **Lab3/** directory of the main course repository. You can find the code at **STT_Assignments** on GitHub.

Laboratory Session 4

Introduction

This lab expanded prior repository mining work by scaling to **multiple heterogeneous open-source projects** and performing a **comparative analysis of Git diff algorithms** (Myers vs Histogram) across file changes. The central goals were:

- Consolidating commit-level and file-level change artifacts from three repositories.
- Extracting dual diff representations (Myers, Histogram) per modified file.
- Constructing a structured dataset linking code snapshots, diff variants, and commit metadata.
- Quantifying discrepancy patterns between algorithms.
- Deriving file type-aware mismatch statistics and visual analytics.
- Outlining a criteria-driven framework for automatic diff algorithm comparison.

Tools

- Python 3: orchestration, data extraction.
- **PyDriller**: commit traversal, file change introspection.
- **git** CLI: raw diff generation with algorithm selection via `--diff-algorithm=`.
- **matplotlib** + **collections.Counter**: statistical aggregation and visualization.
- CSV + filesystem staging: scalable handling of large diff artifacts.

Setup

For mining and analyzing software repositories, I used **PyDriller**, a Python framework that provides easy access to Git metadata. Since Python was already installed and available in the environment, I installed PyDriller using **pip**:

```
1 pip install pydriller
```

To confirm a successful installation, I launched Python in interactive mode and imported the package:

```
1 import pydriller
2 print(pydriller.__version__)
```

This ensured that **PyDriller** was properly installed and ready to be used for repository mining in the lab.

Methodology and Execution

Repository Selection

The first task involved selecting three medium to large sized repository based on specific criteria. For this, I used the *SEART Search Engine for GitHub* to filter repositories.

The criteria I applied were as follows:

- The repository must have a minimum of 1000 commits and maximum of 2500 commits.
- The repository should have at least 25000 stars.

The screenshot shows the SEART Search Engine for GitHub interface. It features a dark theme and a grid of filter categories. The 'General' section includes a search bar, license filter, and topic filter. The 'History and Activity' section has filters for commits, contributors, issues, pull requests, branches, and releases. The 'Popularity Filters' section includes filters for stars, watchers, forks, and codebase size. The 'Date-based Filters' section has filters for creation and last commit dates. The 'Additional Filters' section includes sorting options and repository characteristics like 'Exclude Forks', 'Only Forks', 'Has Wiki', 'Has License', 'Has Open Issues', and 'Has Pull Requests'.

The three distinct repositories (snapshots embedded locally) chosen were:

- **butterknife** (Java / Android annotation binding framework).
- **alpine** (Lightweight JavaScript reactivity library).
- **ninja** (C/C++ build system generator).

These represent varied ecosystems (Python-independent), multiple languages (Java, JS, C/C++), and diverse file structures—providing a richer substrate for evaluating diff algorithm robustness across implementation styles.

Cloning the Repositories

Once the repositories were identified, the next step was to clone it locally for further exploration and analysis. This was achieved using the `git clone` command as shown below:

```
1 git clone https://github.com/JakeWharton/butterknife.git
2 git clone https://github.com/alpinejs/alpine.git
3 git clone https://github.com/0x676e67/ninja.git
```

After cloning, the repositories was available on my local machine

Run Software Tool on the Selected Repositories

Strategy

The objective of this task was to execute `pydriller` across the selected repositories and generate a consolidated dataset of commit–file modifications. For each modified file, metadata such as commit identifiers, file paths, commit message, and diff outputs were extracted and stored in plain text format. Two diff algorithms (`myers` and `histogram`) were executed to capture variation in line-based changes.

Implementation

A single Python script was developed to automate repository traversal, diff extraction, and CSV storage. For each commit and file modification:

- The old and new source codes were dumped into `codes/`.
- Both `myers` and `histogram` diffs were computed using `git diff` and saved into `diffs/`.
- Metadata was appended into a consolidated CSV file.

Listing 9: Consolidation of commit diffs using PyDriller and Git

```

1  # Extract commit metadata and diffs for each repository
2  for repo_path in repo_paths:
3      for commit in Repository(repo_path).traverse_commits():
4          for mod in commit.modified_files:
5              diff_myers = get_diff(repo_path, parent_sha, commit.hash,
6                                  rel_filepath, "myers")
7              diff_hist = get_diff(repo_path, parent_sha, commit.hash,
8                                  rel_filepath, "histogram")
9              writer.writerow([
10                 repo_path,
11                 before_path, after_path,
12                 commit.hash, parent_sha,
13                 commit.msg,
14                 myers_path, hist_path,
15                 discrepancy
16             ])
```

The resulting CSV file (`consolidated_commit_diffs.csv`) captures a unified dataset with the following columns:

- Repository path, old file path, new file path
- Commit SHA and parent SHA
- Commit message
- Paths to Myers and Histogram diff files
- Discrepancy label

This dataset enables downstream analysis of diff consistency and commit-level metadata.

old_file path	new_file path	commit SHA	parent commit SHA	commit message	diff_myers ¹	diff_hist ²
...	...	...	...	...	...	...
...	...	...	...	...	...	...

Discrepancy Flagging

A simple equality check:

$$\text{Discrepancy} = \begin{cases} \text{Yes} & \text{if Myers diff} == \text{Histogram diff} \\ \text{No} & \text{otherwise} \end{cases}$$

To compute file-level differences between commits, a helper function `get_diff()` was implemented. This function acts as a wrapper around the native `git diff` command, enabling consistent extraction of code differences while supporting multiple diff algorithms. The function accepts repository path, parent commit SHA, current commit SHA, target file path, and the diff algorithm (`myers` or `histogram`) as inputs. It constructs a `git diff` command with whitespace and blank line differences ignored. This ensures that only meaningful code changes are captured.

Listing 10: Helper function for diff extraction using Git subprocess calls

```
1 def get_diff(repo_path, parent_sha, sha, filepath, algorithm):
2     cmd = [
3         "git", "diff", "-w", "--ignore-blank-lines",
4         f"--diff-algorithm={algorithm}",
5         parent_sha, sha, "--", filepath
6     ]
7     try:
8         result = subprocess.run(
9             cmd, cwd=repo_path,
10            capture_output=True, text=True,
11            encoding='utf-8', errors='replace',
12            check=True
13        )
14        return result.stdout
15    except Exception:
16        return ""
```

On successful execution, the function returns the raw diff text generated by the specified algorithm. If the command fails due to missing file paths, invalid SHAs, or subprocess errors, the function returns an empty string. This design ensures robustness, allowing the overall pipeline to continue execution even when individual diffs cannot be generated.

Compare Diff Outputs for Discrepancy Analysis

Strategy

The goal of this step was to directly compare the diff outputs obtained via the Myers and Histogram algorithms. Discrepancies indicate structural variation in how the algorithms detect line changes for the same commit.

Implementation

Within the same Python script, a boolean check was embedded:

```
discrepancy = "Yes" if (diff_myers == diff_hist) else "No"
```

This label was appended to the consolidated CSV under a new column named **Discrepancy**. Thus, each commit–file entry carries a clear Yes/No indication of diff consistency.

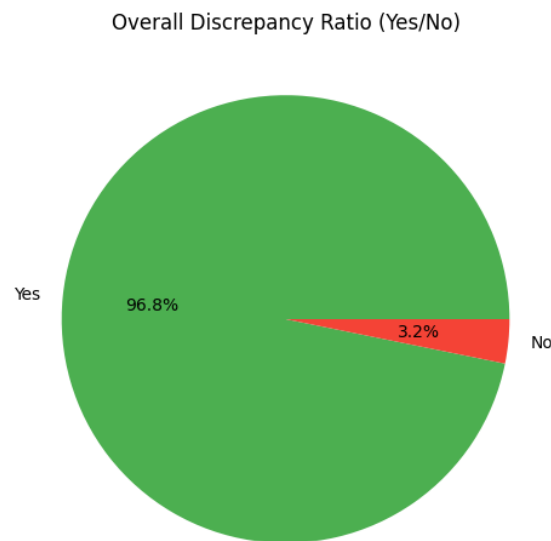
old_file_path	new_file_path	commit_SHA	parent_commit_SHA	commit_message	diff_myers	diff_hist	Discrepancy
...	...	...	...	...	...	...	Yes
...	...	...	...	...	...	...	No

The final dataset highlights cases where both algorithms converge (label = **Yes**) versus diverge (No). These discrepancy patterns can guide further investigation into the robustness of diff algorithms across heterogeneous codebases.

Final Dataset Statistics

A total of **8,705** file-level change rows were processed across all repositories. The comparison of Myers and Histogram diff algorithms yielded the following results:

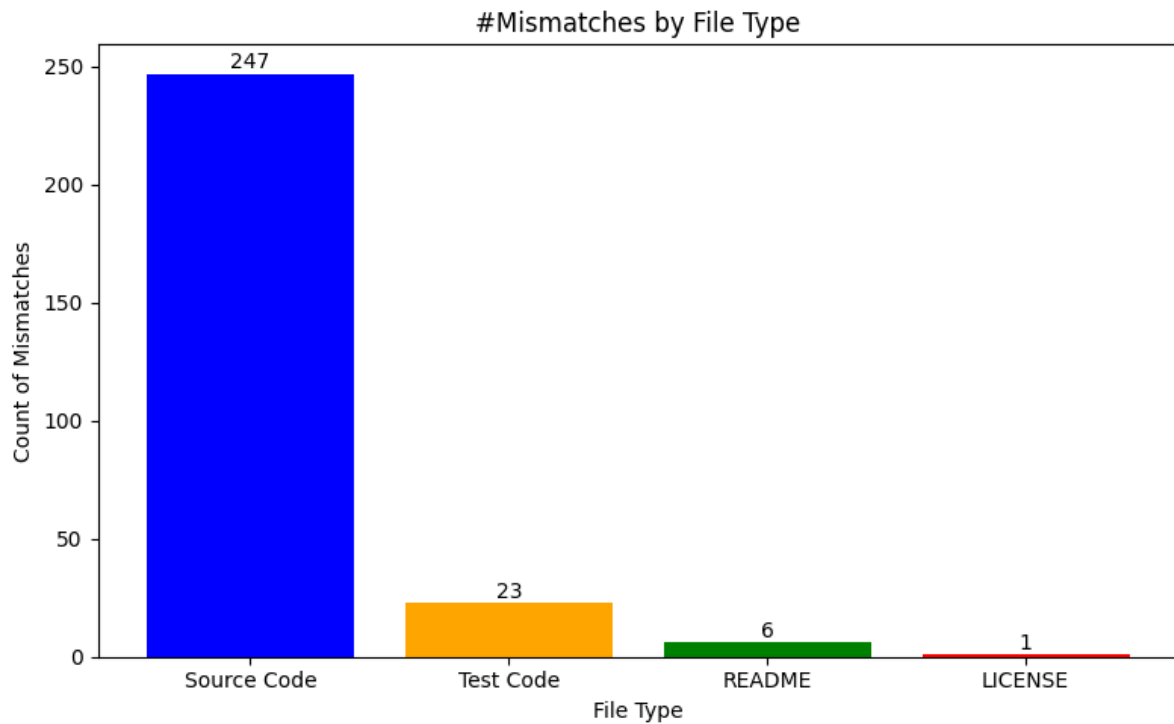
- **Identical outputs (Myers == Histogram):** 8,428 rows (**96.82%**)
- **Different outputs (Myers $\neq$ Histogram):** 277 rows (**3.18%**)



Mismatch Counts by File Type

The breakdown of mismatches (`Discrepancy == 'No'`) by file type is as follows:

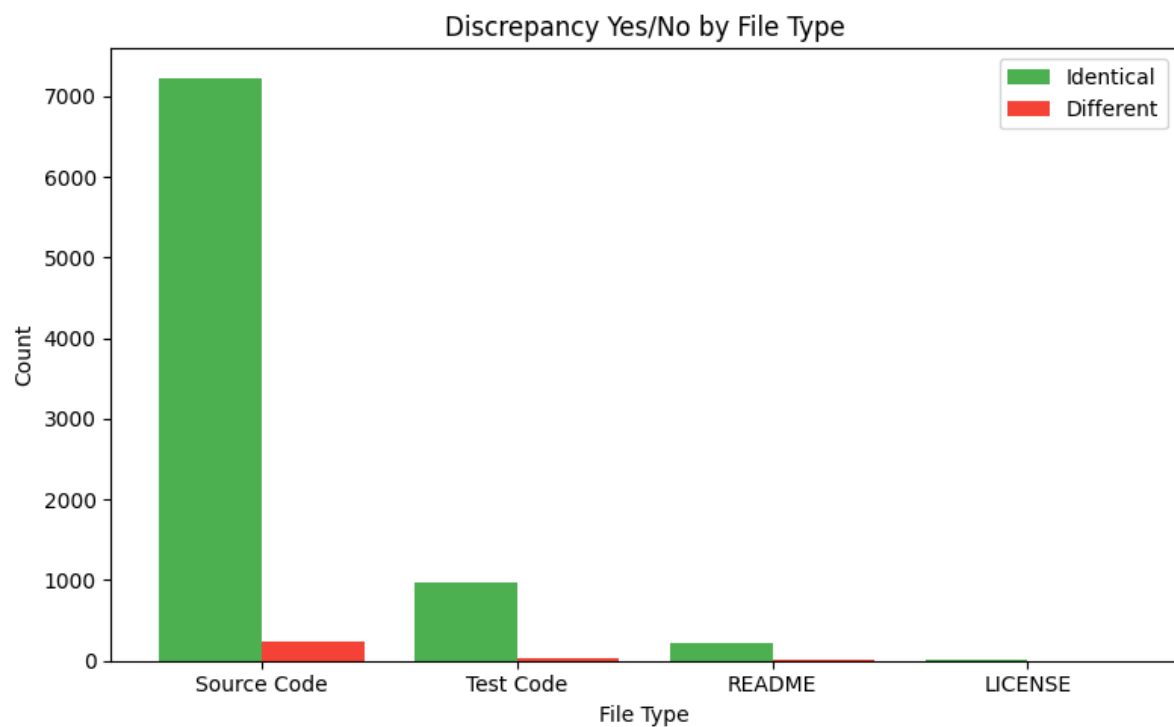
- **Source Code:** 247 mismatches (**3.30%** of source code rows)
- **Test Code:** 23 mismatches (**2.31%** of test code rows)
- **README:** 6 mismatches (**2.68%** of README rows)
- **LICENSE:** 1 mismatch (**14.29%** of LICENSE rows)



Per Repository Outcome

The identical and different outcome rates for each repository are:

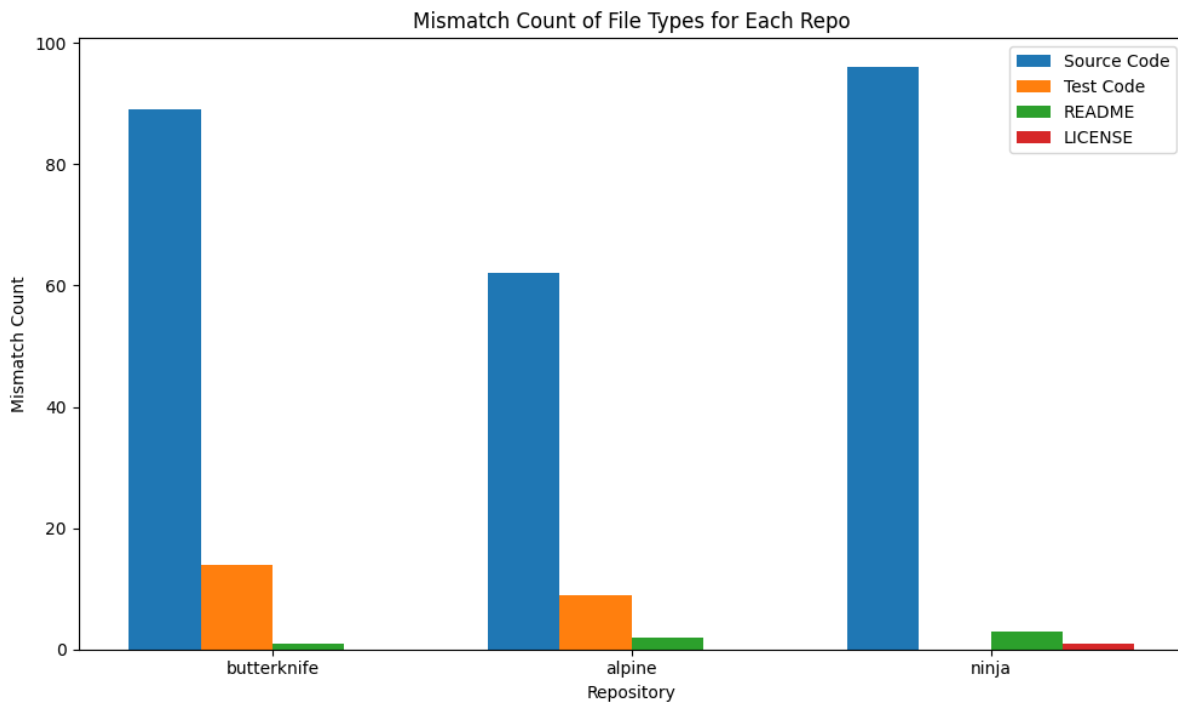
- **butterknife:** 3,027 identical (**96.68%**), 104 different (**3.32%**)
- **alpine:** 1,784 identical (**96.07%**), 73 different (**3.93%**)
- **ninja:** 3,617 identical (**97.31%**), 100 different (**2.69%**)



Mismatches by File Type per Repository

Repo	Source Code	Test Code	README	LICENSE
butterknife	89	14	1	0
alpine	62	9	2	0
ninja	96	0	3	1

Table 1: Mismatch counts by file type for each repository



Interpretation

The results indicate that Myers and Histogram algorithms produce highly similar outputs for the vast majority of file changes (>96%). Mismatches are most frequent in source code files, but the overall rate remains low. The LICENSE file type shows a higher relative mismatch rate, though the absolute count is minimal. Repository-wise, all three projects exhibit consistent patterns, with ninja showing the highest identical rate.

Automatic Evaluation of Diff Algorithm Performance

To objectively determine which diff algorithm—**Myers** or **Histogram**—performed better on the dataset, the following automated evaluation framework was adopted:

Evaluation Criteria

- **Discrepancy Rate:** The primary metric is the proportion of file-level changes where the outputs of the two algorithms differ. A lower discrepancy rate (i.e., more identical outputs) suggests greater consistency and reliability.
- **File Type Sensitivity:** Discrepancy rates are further analyzed by file type (Source Code, Test Code, README, LICENSE) to identify if one algorithm excels in specific contexts.

- **Manual/Oracle Comparison (Optional):** If ground truth or expected diffs are available, each algorithm's output can be compared to this reference for a more robust assessment.

Implementation Steps

1. **Count Mismatches:** For each file-level change, record whether the Myers and Histogram outputs are identical or different.
2. **Aggregate Statistics:** Summarize the number and percentage of mismatches overall and by file type.
3. **Visualization:** Generate bar plots and tables to compare the algorithms across different dimensions.

Decision Rule

The algorithm with **fewer mismatches** or better alignment with expected changes (if ground truth is available) is considered superior for the dataset. Reporting results per file type provides additional nuance, revealing strengths or weaknesses in specific contexts.

Summary

In summary, the automatic comparison of diff algorithms is based on counting mismatches, analyzing results by file type and overall, and visualizing the findings. This approach provides a reproducible and scalable method for evaluating algorithmic performance in software repository mining.

Discussion

Algorithmic Behavior Nuances

Myers optimizes for minimal edit scripts; Histogram, by contrast, prioritizes meaningful block boundary detection in the presence of repeated lines. Divergences frequently emerge in:

- Test fixtures with repeated scaffolding.
- Generated or declarative regions (annotation-heavy Java).
- Reordered blocks or refactoring commits.

Threats to Validity

- **Internal:** Discrepancy flag conflates semantic vs superficial diff differences (equality test is coarse).
- **Construct:** File type inference may mismatch actual modified artifacts.
- **External:** Three repositories, while diverse, may not generalize to, e.g., monorepos or deeply templated codebases.
- **Conclusion:** Absence of human-ground-truth diff quality ratings limits normative claims ("better" vs "different").

Conclusion

Lab 4 operationalized a multi-repository, dual-algorithm diff extraction pipeline producing a reusable, reference-linked dataset. The structured approach enabled:

- Reproducible artifact generation (CSV + external text files).
- Comparative diff analysis without embedding large patches inline.
- Early insights into where algorithmic divergence clusters.

This foundation can directly support subsequent tasks (e.g., semantic enrichment, ML-based diff summarization, or patch quality scoring).

Repository Context

All Lab 4 materials reside under the `Lab4/` directory of the main course repository. You can find the code at `STT_Assignments` on GitHub.